

Part I.

Theory

2. Machine learning

In this section, we introduce the deep learning methods used for this work, such as convolutional neural networks (CNNs) and Fourier neural operators (FNOs).

The terms artificial intelligence (AI), machine learning (ML) and deep learning are used interchangeably in some literature and public media; here I clarify what each one refers to. AI is the broadest field as it encompasses algorithms able to showcase intelligence ^a Machine learning is encompassed within AI, and it is concerned with programs that can enhance their performance on a given task by "learning" from data. Finally, deep learning is one of the most prevalent forms of ML in recent years; this branch of ML is defined by the use of architectures known as deep neural networks (from which it receives the "deep" name), formed by different layers through which the data flows.

^aWhat is considered as showcasing intelligence goes beyond the scope of the thesis. One of the most famous formalisms of such said intelligence is the Turing Test, where a human evaluates a text that could have either originated from a machine or a human. If the machine is capable of fooling the human into thinking a human has written the text, it can be said that the machine shows some intelligent behaviour.

Humanity has long dreamed of machines capable of learning. The earliest records we have of this dream date back thousands of years to Ancient Greece; myths like Talos or Pandora are some of the first recorded mentions of artificial intelligence [MAYOR, 2018, Introduction]. Nowadays, these concepts wouldn't exist outside of fiction, and myths weren't it, but rather the rise in computational power and developed techniques since the 1950s.

The development of artificial intelligence over the years can be summarised in 4 different epochs, which are usually called summers and winters within the literature

[Toosi et al., 2021]. The birth point of AI can be traced back to 1956 at the Dartmouth Summer Research Project. This started a period of great excitement for the field known as "The first AI summer", which lasted until the late 1960s, when, due to overpromises and underdelivered results, major funding was cut, which in consequence started "The first AI winter". During the 1980s, the field reappeared under a new robust approach called "expert systems". This name relates to the idea that during the first AI summer, AI was designed to provide general solutions from basic reasoning premises, similar to how humans and other intelligent beings think and similar to what nowadays is referred to as "foundational models". These algorithms lacked the scalability and generalisation expected, and in contrast, "expert systems" focused on a narrower subset of solutions derived from domain-specific information. This second AI summer wouldn't last long, as hardware manufacturers rejected the specialised needs of such algorithms, and by the late 1980s the "Second AI winter" started. During this second winter, key developments in the field and computational science allowed the recent rise of AI that we are living in nowadays. Some of the key advancements that made AI possible are the development of the backpropagation algorithm and its application to a dataset of handwritten numbers by Yann LeCun [LeCun et al., 1989], the advancements in Big Data and overall the high availability of data due to the internet, and more broadly the increase in computational capabilities due to GPU technologies.

In recent years, machine learning (ML) has become an increasingly important tool across a broad range of scientific disciplines. The combination of growing datasets, advances in computational resources and the ability of models to identify complex relationships has enabled new approaches to data analysis, parameter inference, and numerical modelling. Thus, ML techniques have been adopted in fields such as biology for genomics and protein structure prediction [Alam et al., 2025, Jumper et al., 2021], climate science for weather forecasting and Earth system modelling [Eyring et al., 2024], particle physics for event reconstruction and anomaly detection [Karagiorgi et al., 2021], or materials science for accelerated materials discovery [Butler et al., 2018] to mention a few. Astrophysics has followed this trend, where ML is now routinely applied to problems including source classification, photometric redshift estimation, gravitational-wave detection, cosmolog-

ical parameter inference, and the emulation of computationally expensive numerical simulations [Ting, 2026, Dvorkin et al., 2022, Baron, 2019].

2.1. The smallest neural network: The perceptron

Machine learning (ML) is the field of study concerned with algorithms that improve their performance at a task through experience, represented by data [Goodfellow et al., 2016, Chapter 5]. In defining what learning means, we ended up with the needed ingredients for any machine learning program:

- **A task:** The chosen problem to solve, categorised under classification and regression.
- **A performance measure:** A quantitative value of the performance of the model on the given task.
- **The data:** The dataset used for training the model. When the data is labelled, and these labels are used to train the model, the algorithm is referred to as supervised. Whereas if the data is unlabelled, or simply not used during training, it is referred to as unsupervised.

Machine learning encompasses a broad range of algorithms, including decision trees, support vector machines, kernel methods, and neural networks. Among these, deep learning has emerged as one of the most successful approaches for learning complex, nonlinear mappings from high dimensional data.

Deep learning is the branch of machine learning focused on neural networks composed of concatenations of linear functions interleaved with nonlinear activation functions. The simplest building block for a neural network architecture¹ is known as a perceptron. A perceptron is a function that transforms inputs $\mathbf{x} = [x_0, x_1, \dots, x_p]^T$ ² into a scalar output y . This function has the following parameters: the weights $\mathbf{w} \in \mathbb{R}^{p \times 1}$ and the bias $b \in \mathbb{R}$,

¹Architecture and model both refer to neural network programs with some subtle differences. Architecture is the combination of perceptrons and different methods used that take the input to the output. In contrast, a model is an instance of such an architecture with the parameters learned from training.

²Throughout this work, we are going to use the column vector standard

and an activation function σ :

$$y = \sigma(\mathbf{w}^T \mathbf{x} + b), \quad (1)$$

a scheme of a perceptron can be seen in Figure 2.1.

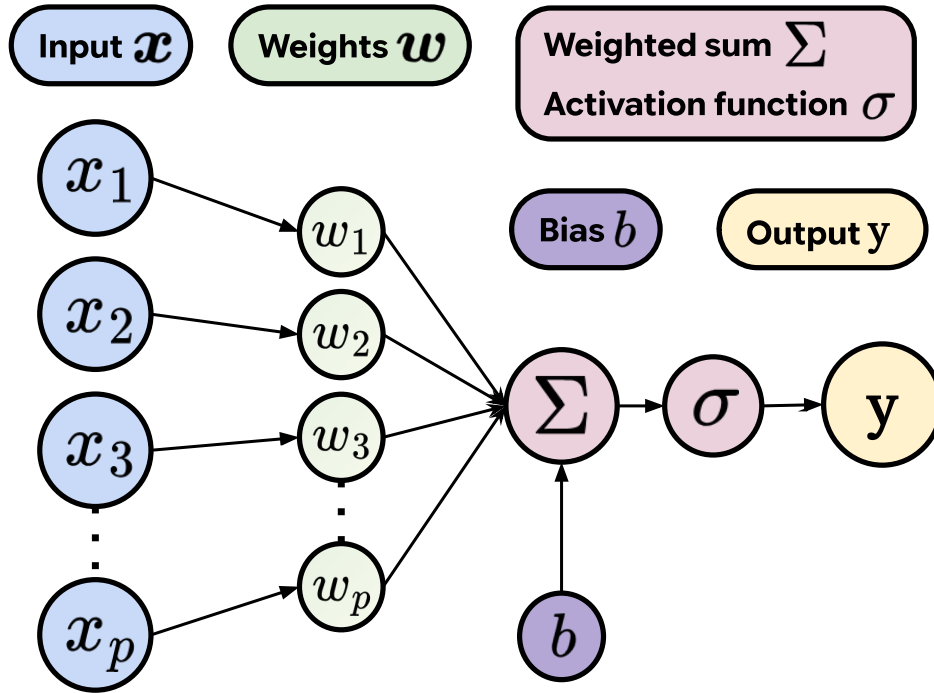


Figure 2.1: Scheme of a basic perceptron. A weighted sum is performed on an input $\mathbf{x}$ with an extra sum term, the bias. The sum output is input to a nonlinear activation function σ to obtain the output y .

When the activation function is the identity mapping $\sigma(x) = x$, the perceptron is reduced to a linear regression architecture. When σ is chosen as the sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$ the architecture receives the name of logistic regression. These two architectures are categorised in the broader categories of regression and classification, respectively. The choice of activation function σ would require work on its own. As will be shown in later sections, to leverage the ability of the models to "learn" complex and nonlinear behaviour from data, we must enforce the nonlinearity of the activation function.

Let's consider the aforementioned example of logistic regression, where the prob-

ability of the input being in one of two different classes (1 or 0) is predicted. Such a model can be written as:

$$p(y = 1|\mathbf{x}, \mathbf{w}, b) = \frac{1}{1 + e^{-\mu(\mathbf{x}|\mathbf{w}, b)}} \quad \mu(\mathbf{x}|\mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b, \quad (2)$$

where y is the label (1 or 0); as such the probability $p(y = 0|\mathbf{x}, \mathbf{w}, b) = 1 - p(y = 1|\mathbf{x}, \mathbf{w}, b)$ and μ receives the name of logit or pre-activation.

With a mathematical description of our perceptron, we are left to describe how to set the weights such that they produce accurate predictions for our problem. To achieve this, a metric to evaluate the performance of our perceptron is needed. Given a dataset $D = \{x_i, y_i\}_{i=0}^N$ of N pairs, where x_i denotes the input feature vector and y_i the corresponding target value, a performance measurement can be written. A standard choice for regression problems is the mean squared error 3:

$$MSE = \frac{1}{N} \sum_{i=0}^N (y_i - \hat{y}_i)^2, \quad (3)$$

where $\hat{y}$ is the predicted output from our perceptron. The performance measure choice depends on the problem and an in depth study is beyond the scope of this work. Standard choices are the mean squared error for regression problems, where the predicted output is in the continuous range, and the cross-entropy loss when predicting discrete labels, like in classification. Assuming the data is sampled from an underlying data generating distribution p_{data} , the performance measurement can be expressed as an expected value over the distribution, known as the risk J :

$$J(\theta) = \mathbb{E}_{(x,y) \sim p_{data}} L(f(x; \theta), y), \quad (4)$$

where L is the loss function, which quantifies the error per data point, $f(x; \theta)$ is a deep neural network and θ are the parameters of the network. This definition is written under the assumption of unregularised supervised training; therefore, the target data y is used in the loss function.

Training a neural network consists of finding the parameter vector θ that minimises the risk function. Since the expectation over the true data generating distribution p_{data} is generally inaccessible, the risk is approximated by its empirical version and computed over the available training dataset. The optimisation problem is typically solved using gradient optimisation methods, usually stochastic gradient descent (SGD) and its variants, which iteratively update the network parameters in the direction of decreasing loss.

Although a perceptron can model nonlinear relationships through its activation function, it is still limited. A single perceptron computes a single affine transformation followed by a nonlinear activation and thus can only represent relatively simple decision boundaries. Many practical problems require more complex representations than can be achieved with a single computational unit. To exploit the capabilities of the perceptron, we need to use more than one at a time, which motivates the introduction of multilayered perceptrons.

2.2. Multilayered perceptrons

Multilayered perceptrons (MLPs) are organised in layers, with each layer composed of several individual perceptrons, where the perceptrons of the previous layer are all connected to the perceptrons of the next layer. These layers are called fully connected layers. Each layer receives as input the output of the previous layer (the input in the case of the first layer, also known as the input layer). Any layer that is not the input or output layer is known as a hidden layer. Each layer is composed of $d^{(l)}$ perceptrons where l signifies the layer (starting from 0 at the input all the way to the total number of layers L). A scheme of a multilayer perceptron can be seen in Figure 2.2. The mathematical description of an MLP can be written as

$$\begin{aligned} \mathbf{h}^{(0)} &= \mathbf{x} \\ \mathbf{h}^{(l)} &= \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \quad l = 1 \dots L \\ \mathbf{h}^{(L)} &= \mathbf{y} \end{aligned} \tag{5}$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{d^{(l)} \times d^{(l-1)}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{d^{(l)} \times 1}$. Here we assume σ is the same throughout the network; this is not necessarily true, as depending on the expected output, a design choice or inductive bias could be introduced through the last activation function. For example, in the case of working with physical quantities where non-negativity applies, such as pressure or density, a suitable choice of final activation function could be a Rectified Linear Unit (ReLU), which is defined by:

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{if } x \leq 0, \end{cases} \quad (6)$$

the ReLU is one of the most used activation functions in machine learning for its stability as we will see later in the backpropagation section.

The MLP is one of the earliest developed deep neural architectures³. Its expressive power lies in the nonlinearity of the activation function, as without it there would be no need to use the layered structure, as the network would reduce to a simple linear transformation. These nonlinearities make the optimisation problem of training a deep neural network highly non-convex. In classical optimisation problems, second order methods such as Newton's method can be applied, these methods can't be applied to neural networks due to the large number of parameters, typically in the millions. Instead, training is performed with first order gradient optimisation methods.

³MLPs might seem less prominent nowadays due to more modern architectures such as Diffusion models or Transformers. It's important to note that within transformer blocks, a fully connected layer is used afterwards, signifying the importance this architecture entails.

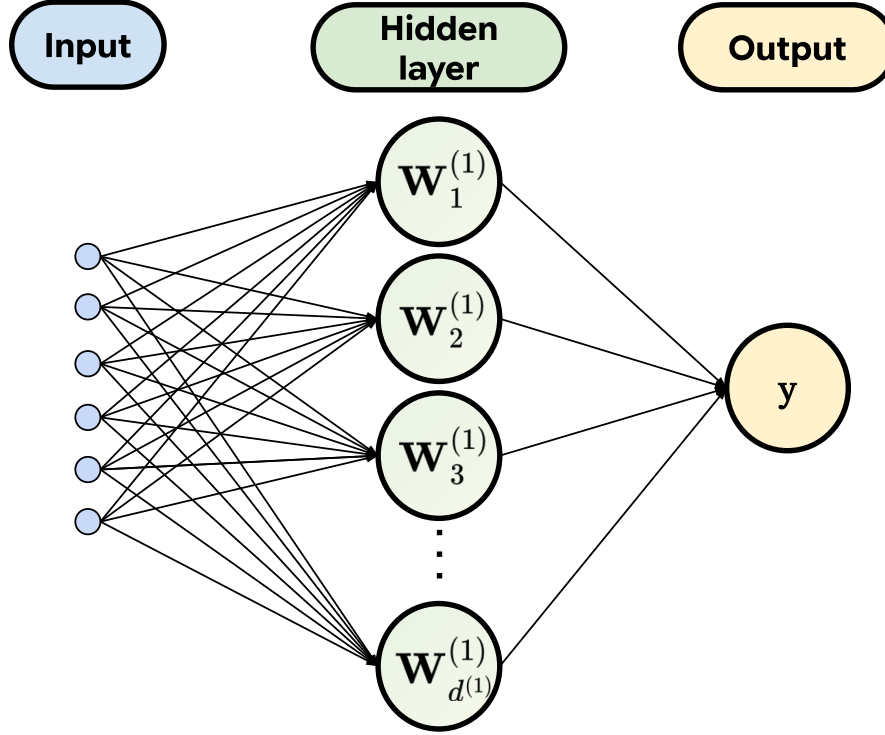


Figure 2.2: Scheme of a multilayer perceptron with 1 hidden layer. For visual simplicity, the linear weight multiplication, the bias, and the activation function are grouped in the perceptron. The architecture maps an input vector $\mathbf{x} \in \mathbb{R}^{d^{(0)} \times 1}$ to an output $y \in \mathbb{R}$ through a hidden layer composed of $d^{(1)}$ perceptrons, corresponding to the weight matrices $\mathbf{W}^{(1)} \in \mathbb{R}^{d^{(1)} \times d^{(0)}}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{1 \times d^{(1)}}$, which in the image $\mathbf{W}^{(2)}$ is embedded in the output perceptron y

Lastly, it is worth motivating the use of neural networks through the theoretical framework. This is done through the Universal Approximation Theorem. The theorem has several variations, and here we present the original formulation, which deals with a network consisting of a single hidden layer of arbitrary width. Given such a network and a sigmoid activation function, the network can approximate up to precision ϵ any arbitrary function. Written formally, the theorem looks like:

Theorem 1 [Hornik et al., 1989] *Let $\mathbb{X}$ be any compact subset of $\mathbb{R}^n$ and let σ be any*

sigmoid activation function. Then the finite sum of the form

$$f_{\text{NN}}(x) = A_2 \sigma(A_1 x + b_1) = \sum_{j=1}^{m_1} a_{2j} \sigma(A_{1j} x + b_{1j}) \quad (7)$$

is dense in $\mathbb{X}$.

In other words, given any function $f : \mathbb{X} \rightarrow \mathbb{R}$ and any $\varepsilon > 0$, there exists a finite sum f_{NN} of the form in Eq. (7) such that

$$|f(x) - f_{\text{NN}}(x)| < \varepsilon \quad \text{for all } x \in \mathbb{X}.$$

Further in [Leshno et al., 1993], the theorem was proven for any non-polynomial arbitrary activation function. This theorem is also studied in the direction of arbitrary depth (arbitrary number of layers) with a single neuron instead of arbitrary width. These theorems give theoretical motivation to the use of neural networks to approximate any unknown function. For further reading on universal approximation theorems, we refer to [Augustine, 2024].

2.3. Optimization of neural networks

The loss landscape of a deep neural network is highly non-convex; traversing it to find optimal parameters for a given model is the most difficult and central task in neural network training. The parameters should yield accurate predictions on data outside of the training distribution. These optimal parameters are given by minimisers of the risk:

$$\hat{\theta} = \arg \min_{\theta} J(\theta) = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim p_{\text{data}}} [L(f(x; \theta), y)] \quad (8)$$

Since p_{data} is unknown, the risk can't be computed analytically. Therefore, we need to substitute the expected value with the empirical version:

$$\hat{J}(\theta) = \mathbb{E}_{(x,y) \sim \hat{p}_{\text{data}}} L(f(x; \theta), y) = \frac{1}{N} \sum_{i=1}^n L(f(x_i; \theta), y_i), \quad (9)$$

where the expected value was substituted with an averaged sum over all the individual losses through the data $D = \{x_i, y_i\}_{i=0}^N$, this formula is known as the cost function or the empirical risk.

The empirical risk function and the MSE are motivated by the maximum likelihood estimation (MLE) method. The maximum likelihood estimate of θ is the value that maximises the likelihood of the parameters given the data points :

$$\mathcal{L}(\theta) = \prod_{i=1}^m p_{\text{model}}(y_i | x_i; \theta), \quad (10)$$

the best estimators for the parameters are therefore given by:

$$\hat{\theta} = \arg \max_{\theta} \mathcal{L}(\theta) = \arg \min_{\theta} [-\log \mathcal{L}(\theta)] = \arg \min_{\theta} \left[-\sum_{i=1}^m \log p_{\text{model}}(y_i | x_i; \theta) \right]. \quad (11)$$

This last form is known as negative log likelihood (NLL) and is preferred over the original form due to numerical underflows: as probabilities go from 0 to 1, if the sample data is large enough, multiplying them would result in a number impossible to represent with 64-bit architectures. To connect back to the MSE, if the conditional model distribution is assumed to be Gaussian, the NLL takes the form (up to constants) of the MSE.

Due to the high dimensionality of θ and non-convexity of the hypersurface defined by $\hat{J}(\theta)$, analytical solutions are non-existent. Therefore, we need to rely on first-order methods; the main method used in machine learning is Stochastic Gradient Descent (SGD), which is a stochastic approach to gradient descent. Gradient descent methods are iterative first order optimisation methods in which the parameters are optimised in the negative direction of the gradient in the hypersurface defined by $\hat{J}(\theta)$.

To make use of gradient descent, the cost function must be differentiable. Under that

assumption, the gradients are given by:

$$\nabla_{\theta} \hat{J}(\theta) = \frac{1}{N} \sum_{i=1}^n \nabla_{\theta} L(f(x_i; \theta), y_i), \quad (12)$$

In later sections, we will comment on how these gradients are calculated through the forward computation. Given the gradients, the parameters of the neural network can be updated following the gradient descent formula:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t), \quad (13)$$

where t signifies the iteration index, also known as epoch number in machine learning, and η the learning rate. The parameters are updated in the opposite direction of the gradient as to decrease the cost function. The stopping mechanism is then determined by the maximum number of epochs set beforehand or more sophisticated methods like plateau detection, known in the machine learning literature as early stopping.

Stochastic gradient descent adds stochasticity to gradient descent by not training on the full set of training data in each step, but instead choosing a reduced batch and training on it. The stochasticity introduced by optimisation dynamics adds noise to the gradient updates. This noise can help the optimiser escape local minima and go towards wider minima that generalise better over unseen data. Another advantage of SGD is merely computational: the reduced data reduces the cost per iteration from $\mathcal{O}(N)$ to $\mathcal{O}(n')$ where N is the dataset size and n' is the batch size, making this approach easier to scale for larger datasets.

In this subsection, the basic pieces for a machine learning training framework were introduced. Together, these define an algorithm written in 1, note that we wrote the SGD version as it is more commonly used than basic GD.

Algorithm 1 Stochastic Gradient Descent for Empirical Risk Minimisation

Require: Dataset $D = \{(x_i, y_i)\}_{i=1}^N$, model $f(x; \theta)$, loss L , learning rate η , batch size n' , number of epochs T

- 1: Initialize parameters θ_0
- 2: **for** $t = 0, \dots, T - 1$ **do**
- 3: Sample a mini-batch $B_t \subset D$ with $|B_t| = n'$
- 4: Compute predictions $\hat{y}_i = f(x_i; \theta_t)$ for $(x_i, y_i) \in B_t$
- 5: Compute mini-batch loss:

$$\mathcal{L}_t = \frac{1}{n'} \sum_{(x_i, y_i) \in B_t} L(\hat{y}_i, y_i)$$

- 6: Compute gradient $g_t = \nabla_{\theta} \mathcal{L}_t$
 - 7: Update parameters $\theta_{t+1} = \theta_t - \eta g_t$
 - 8: **end for**
 - 9: **return** $\hat{\theta} = \theta_T$
-

2.4. Backpropagation

Neural network training relies on the computation of derivatives of the empirical risk function to optimise the networks. The methods for derivative computations can be aggregated in 3 classes, [Baydin et al., 2018]:

- **Symbolic differentiation:** The derivative is computed analytically via the well known differentiation rules. There are computational algorithms that perform these computations (i.e., Matlab, Mathematica, etc.). There are two main drawbacks to this approach in a neural network setting: first, the need to write your model as a closed-form expression, thus limiting the use of control flow. Secondly the complexity and intermediate values required for the final result grow as the networks grow, therefore it poses a limit on the scale of the networks, this is known as *expression swell*.
- **Numerical differentiation:** Through exploiting the definition of derivative, numerical methods have been developed to allow for the computation of approximate gradients, such as finite difference methods (forward, backwards or central). The main drawback of these methods is their inaccuracy due to their approximate

nature and sensitivity to the step size. In particular, for neural networks, such a computational framework would require two forward passes for each parameter. Therefore, this limits the scalability of the algorithm to large datasets and networks.

- **Automatic differentiation:** Provided a computational graph, the derivative is computed by traversing this graph in either the forward or reverse direction. In doing so, the intermediate partial derivatives (tangents in forward mode and adjoints in reverse mode) at each node of the computational graph are computed with respect to the connected nodes (the children for the reverse mode and the parents for the forward mode). The derivative value is then obtained by combining the computed intermediate values following the chain rule. This method provides an accurate gradient (up to the floating point precision) and high computational performance.

Automatic differentiation (AD), although not adopted at first in machine learning, as shown in [Domke, 2009], has proven to be the most effective gradient computational method, as it provides the same accuracy as symbolic differentiation while achieving better performance than symbolic and, in some cases, numerical differentiation.

Further going into detail of AD, the computation of the derivative of the output y with respect to the inputs x_i can be performed in the forward or reverse modes (in this example we are showing how AD works with a scalar output as in ML this is mainly the case, this method is easily extendable to the case where the output lives in a larger space).

If now we want to compute the derivative of y with respect to x_i in forward mode AD computation, we must start by assigning each of the intermediate values v_i a value:

$$\dot{v}_i = \frac{\partial v_i}{\partial x_1}, \quad (14)$$

known as the tangent. Then, by use of the chain rule, we can write the tangents as:

$$\dot{v}_i = \sum_{j \in \text{parents}(i)} \dot{v}_j \frac{\partial v_i}{\partial v_j}, \quad (15)$$

where parents means the predecessors of the node v_i in the computational graph. By computing the tangents along the computation, the final derivative can be computed. In this case, the derivative starts from the input towards the output and has to be done for each input.

In the case of reverse mode, the intermediate value assigned to each node differs from the forward mode and now takes the shape,

$$\bar{v}_i = \frac{\partial y}{\partial v_i}, \quad (16)$$

these are known as adjoints, and by using the chain rule we can define them in terms of their children nodes:

$$\bar{v}_i = \sum_{j \in \text{children}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}. \quad (17)$$

The calculation is then done in a two-step fashion: first, the forward pass of the graph has to be done before the derivative calculation (instead of in forward AD, where they are done simultaneously). In doing so, the intermediate values at each node have to be stored, these are called primals. On the second step, starting now from the output, the derivatives are calculated via propagating the adjoints in the reverse direction (hence the name).

Comparing both methods, we can first comment on the shortcomings of forward mode compared to reverse for its application in machine learning. The main drawback lies in the need to perform the computation for each input, whereas in reverse mode this is done once per output. Thus, reverse mode is significantly less costly time wise, although highly costly memory wise compared to forward, as the primals must be stored during the forward pass. To alleviate the memory constraints, methods such as checkpointing or rematerialisation are used. In these methods, not all the primals are saved during the forward pass. Still, only a selected few are saved during the backward pass, the missing intermediate values are computed using the selected few stored primals. In this way, we alleviate memory constraints while trading it off for computational time.

To enable gradient computation, several computational frameworks have been developed

over the years, such as PyTorch, TensorFlow, and Jax. These libraries allow the user to obtain derivatives from automatic differentiation by constructing the computational graph of a given mathematical operation. This abstraction relieves the user from manually deriving and implementing the gradient expressions, while also enabling optimised implementations on CPUs and GPUs.

Lastly, it should be noted that when expanding either gradient computation equation, forward or backwards, what ends up appearing is the product of a long series of derivatives. These derivatives can either be larger or smaller than 1. If enough derivatives have magnitude larger than 1, the final value will explode; conversely, enough derivatives smaller than 1 in magnitude can make the gradient vanish. This poses a challenge for deeper neural network architectures, in which gradients tend to vanish/explode. To overcome this, several techniques to regularise training are used, such as activation function selection, skip connections, normalisation, or gradient clipping. One of them is the selection of activation functions, for example, the ReLU was introduced in Equation 6, whose derivative is equal to one for positive inputs, which helps preserve gradients during backpropagation and alleviates the vanishing gradient problem.

2.5. Advanced and practical training techniques

In the previous section, we introduced the basic elements that constitute neural network training. In this section, we go into detail on certain advanced topics of machine learning training that are used across most training frameworks nowadays or that are used in our project.

2.5.1. Adaptative optimisers: ADAM

While simple gradient descent takes a first-order approximation, the optimiser introduced in [Kingma and Ba, 2017] adapts the learning rate by estimating the first and second moments of the gradients. Hence the name adaptive moment estimation, ADAM. The algorithm differs from standard gradient descent by using the gradient (m_t) and the

squared gradient (v_t) moving averages. These follow the formulas:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \end{aligned} \tag{18}$$

where two new hyperparameters were introduced β_1 and β_2 which determine the decay rate of the moving averages and g_t^2 indicates the element wise square ($g_t \odot g_t$).

As the moving averages are initialised to 0s, the updates are heavily biased at the early stages of training. To correct the bias, the moving averages are weighted in the following way:

$$\begin{aligned} \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t}. \end{aligned} \tag{19}$$

Putting everything together, the Adam parameter update formula is:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \tag{20}$$

where ϵ is a constant to ensure safe division. Optimisers like Adam and AdamW are the most common in machine learning due to their proven performance in experiments. The physical intuition behind it is a boulder rolling down a hill. Where optimisers like basic gradient descent didn't account for accumulated steps in the same direction (the physical momentum of the boulder in our analogy), Adam, by keeping two moving averages, can account for those and retain a stable and directed gradient direction towards wider minima. Despite their rapid convergence and robustness, Adam and AdamW have been observed to produce models that generalise worse than SGD [Reddi et al., 2019]. Although its stochastic variant, stochastic Adam, has recently been found to achieve the same or better performance than SGD [Tang et al., 2025]. Thus, to achieve better performance, it is recommended to either use a stochastic version of Adam or use Adam at the early stages of training, then switch to SGD at later stages.

2.5.2. Learning rate and scheduling

Fixed learning rates are undesirable as the motion on the loss landscape can either overshoot or get trapped in local minima. There are several techniques that, during training, modify the learning rate value depending on the dynamics of the loss, such as the prior introduced SGD and ADAM. Another flavour of learning modifiers are schedulers, which change the learning rate value across training through a fixed formula. Typical shapes for schedulers include decaying schedulers like exponential or cosine decays, or cyclical schedulers such as cosine annealing [Loshchilov and Hutter, 2017]. For a further analysis of learning rate schedulers, we refer the reader to [Defazio et al., 2024].

2.5.3. Data split

It is assumed that the training data has been independently and identically distributed from the data generating distribution p_{data} . With access to sufficiently large amounts of data, the empirical distribution approaches the underlying data generating distribution. As this is not typically the case and we have to work with limited data, be it due to computational or complexity constraints, or lack of data, the training data represents a subset of the original data distribution and, as such, could be biased. A consequence of training with limited data is the possibility of overfitting or underfitting. Overfitting means the model adapts too closely to the training data. On the contrary, underfitting occurs when the model is not capable of capturing the underlying relations in the training data. Both lead to poor performance on unseen data, i.e., poor generalisation.

This issue can not be circumvented in trivial ways and has to be solved through smart architecture and training decisions, as well as other techniques such as data augmentation. One key consequence of such data limitations is how the available data is treated and how the trained models are evaluated against it. A naive approach is to train on the full dataset and then test performance on the same data; this leads to meaningless performance measurements, as the network is evaluated on the same data it trained with. Therefore, the common practice is to partition the available dataset into three subsets:

- **Training:** Used for and during training to optimise the network parameters. Could be stochastically sampled in methods like SGD. Typically represents around 80-70 % of the total available data.
- **Validation:** Used during training to measure the performance in real time against never-used during training data. The performance measurement obtained with this subset already represents a more valid measure than with the training data. Thus such a performance measure is used during real time training to evaluate the evolution of the network's performance and can be used in combination with early stoppers. Typically represents 10-15 % of the total available data.
- **Test:** Used after training to give a final and more definite performance measure than with the validation subset, as the validation subset was used in some way during training and although it consists of never-seen data it still could be bias optimised through methods such as early stoppers. Therefore, the test dataset should only be seen by the network once to provide an unbiased estimate of the model's performance. Typically represents 10-15 % of the total available data.

Due to convenience or data limitations, machine learning frameworks sometimes use training data that originates from simulations or different domains than the ones aimed to predict. Assuming that, after training, the final model performs as expected on the training domain, this does not ensure similar performance on the target domain, because the underlying distributions may differ. Thus, the model has to undergo what is known as transfer learning [Zhuang et al., 2020], where the model is finetuned to adapt to the target domain instead of the source domain on which it was trained.

2.5.4. Physically informed neural networks

Physical systems are constrained by conservation and symmetry laws encoded by partial differential equations. Training a neural network on such a system leads to physically aware neural networks with sufficient data. As this is typically not the case, we must enforce some of these constraints within the network or the training procedure. Such

networks are known as physically informed neural networks (PINNs) and achieve data-driven and physically informed networks through feature engineering, model constraints and extra residual terms to the loss [Farea et al., 2024] :

- **Feature engineering:** Knowledge of the physical quantities that rule the study domain is useful to extract and selectively choose the features the model should take as input to avoid multicollinearity. This approach can also be done the other way: we could create new features as a combination of others that convey greater physical meaning. Either way, feature engineering allows for physics informed data even before the model has seen it. ⁴
- **Model constraints:** Model architectures entail their own constraints by design, be it translation as in the case of convolutional neural networks (CNNs) or rotation in the case of graph neural networks (GNNs). By choosing the right architecture, physical constraints can be addressed from the outset.
- **Extra residual terms:** Where a standard choice for loss function L in machine learning, like the mean squared error, might work for most problems, it does not capture the constraints and physical meaning of a system. To do so, a widely used flavour of PINNs is to add an extra term to the loss function which encodes some physical constraint, usually in the form of a PDE or a conservation law.

2.5.5. Hyperparameter optimization

In the prior sections, several methods and techniques have been introduced to train a neural network. It should come to the attention of the reader that such a framework depends highly on the values of the parameters that affect the training before it even started (known as hyperparameters). Hyperparameters include the learning rate and ADAM parameters, the model used and its parameters, the early stopping patience, the learning rate scheduler, the mini-batch size... It's a balancing effort to set the values of the parameters correctly for stable training. Standardised values are a good starting

⁴Although not explicitly feature engineering, data augmentation methods that aim to enforce other constraints (i.e. augmenting a dataset of images with rotated images, as we know the model should be rotation invariant) could be seen as a way of embedding constraints within the data.

point, still, methods to finetune further include grid searching, random searches, or Bayesian-based methods such as Gaussian processes or Tree-Structured Parzen methods.

Grid search explores the hyperparameter space in a grid fashion, such an approach scales prohibitively with the number of parameters and may underperform against most advanced methods in convergence speed. Similarly, random searches, although useful as a first approach for a small hyperparameter space, may be outperformed in convergence speed by more advanced methods. On the contrary, Bayesian optimisation methods offer a more efficient traversal of the parameter space, although their design and application are considerably more complex. These methods, together with pruning strategies for training instances that don't seem promising, are widely used in modern training frameworks and are grouped under software libraries such as Optuna [Akiba et al., 2019]. Optuna provides an easy-to-use API and support for hyperparameter optimisation within Python; other alternatives exist, such as Ray Tune [Liaw et al., 2018], which specialises in parallelisation.

2.5.6. Skip connections

Skip or residual connections alleviate the vanishing gradient problem by introducing an identity mapping that provides a direct path for the gradient and the computation; they were popularised by the ResNet architecture introduced in [He et al., 2015]. As an example, consider a network composed of 2 differentiated parts f_1 and f_2 , connected through a skip connection:

$$f_{NN} = f_2(x + f_1(x)), \quad (21)$$

applying the chain rule yields

$$\frac{\partial f_{NN}}{\partial x} = \frac{\partial f_2}{\partial (x + f_1(x))} \left(\frac{\partial f_1}{\partial x} + I \right). \quad (22)$$

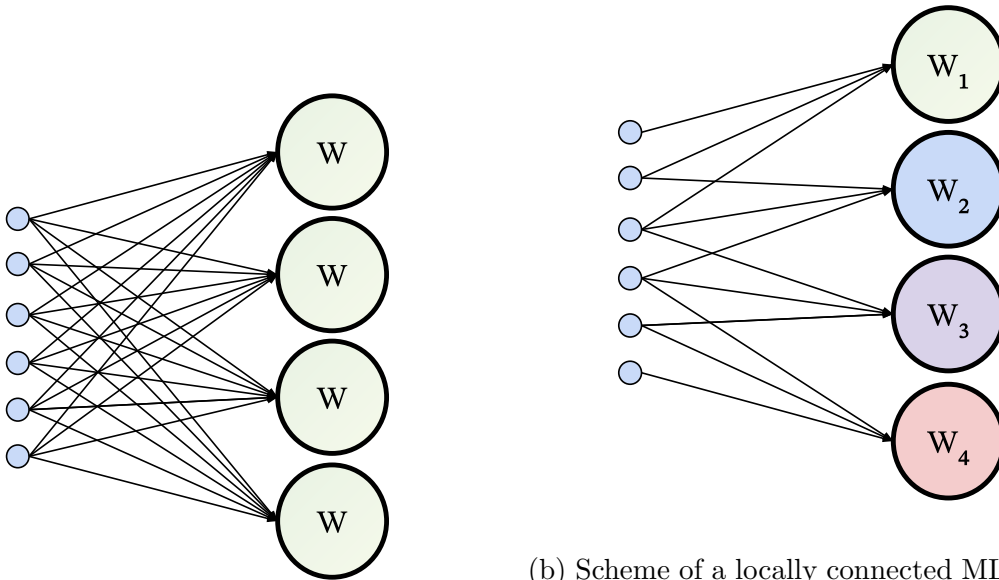
The identity term I provides a direct path for the propagation of the gradient. Therefore, even if the Jacobian of f_1 becomes very small, the gradient can still propagate through

the skip connection, mitigating the vanishing gradient problem while also allowing information to flow more easily through the network.

2.6. Neural network architectures

In the following, we introduce a few neural network architectures that are used throughout this work.

2.6.1. Convolutional Neural Networks (CNNs)



(a) Scheme of a weight sharing MLP, the same weight matrix is used for each of the perceptrons.

(b) Scheme of a locally connected MLP; each perceptron is only connected to 1 input from the previous layer and its neighbours. The number of neighbours depends on the kernel size, which in the figure is 3.

Figure 2.3: Schemes of both a weight sharing and locally connected MLP. By combining these two approaches, the number of parameters required is highly reduced, and the network applies a convolution at the input from the previous layer, thus receiving the name Convolutional Neural Network.

Multilayered perceptrons consist of fully connected layers, these layers are expensive in parameter count, as each one requires $N = (n_{in} + 1) \times n_{out}$. One way to reduce

the number of parameters is by using locally connected NNs or weight sharing NNs. Locally connected NNs reduce the number of parameters by connecting 1 input and its neighbours to an output. In a 1 dimensional case it would be 3 inputs (1 and 2 neighbours) for 1 output. Weight sharing NNs apply the same weight to all the perceptrons in a layer. Both architectures are represented in Figure 2.3. By combining both approaches, the number of parameters is reduced to $N = 3 + 1$ including the bias. Thus, the network applies a convolution to the input from the previous layer, thus the name Convolutional Neural Networks (CNNs).

It should be noted that, although not an explicit requirement, this architecture is order dependent and thus it's usually best applied when the problem at hand has a natural ordering, as is the case of 2D images. Focusing briefly on the 2D case, as it's the most intuitive and most used, we can see in Figure 2.4 the application of the kernel. Given an image I of pixel values $I(i, j)$ and kernel K of values $K(l, m)$ and size $K_h \times K_w$ the resulting output C at the point (x, y) is given by:

$$C(x, y) = \sum_{l=0}^{K_h-1} \sum_{m=0}^{K_w-1} I(i+l, j+m) K(l, m) \quad (23)$$

Widely extended kernel size choices for 2D images are 3x3 or 5x5, although any size can be used. Another important parameter in CNNs is the stride, which defines the step size of the kernel application; together with padding, it defines the size of the output layer. For example, if the input image is 128x128 and we use a kernel of size 5x5 with stride 2 and no padding, the output will result in an image of size 62x62, given by the formula $\lfloor \frac{N-K+2P}{S} \rfloor + 1$. When applying CNNs to 2D images, a per channel approach is used; therefore, a kernel is defined for each channel output. Furthermore, more channels can be created, increasing the dimensionality in the hidden layers. Lastly, it's important to note that, as the convolution operation is shift equivariant, the whole model is. Consequently, CNNs are not only useful due to their ease of memory but also due to their enforced architectural constraint.

CNNs have been successful in applications such as image recognition and classification [He et al., 2015], medical image analysis [Ronneberger et al., 2015], super-resolution

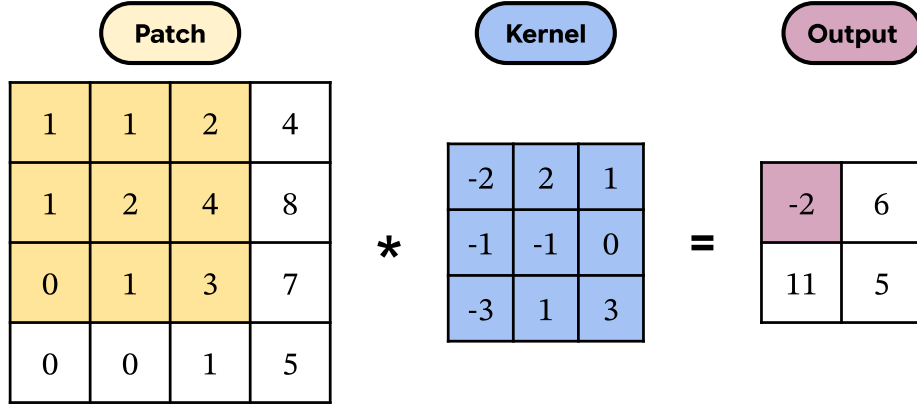


Figure 2.4: Example of a convolution application in the 2D case.

[Zhang et al., 2018] or astronomical analysis [Hayat et al., 2021]. In recent times, CNNs have been widely outperformed by visual transformer architectures. Nevertheless, CNNs still pose a useful solution for certain applications due to their lightness and good performance with small datasets [Pasupuleti, 2025].

2.6.2. Neural operators

The neural networks introduced up to this point act as approximators for discrete functions between finite dimensional Euclidean spaces. This approach, although proven useful, is not mathematically grounded and depends on the discretisation at hand. In [Lu et al., 2021], the neural operator architecture was proposed, where, instead of applying the network to discretised inputs and outputs, the network learns an operator between two function spaces, therefore removing the limitation of discretisation. Examples of operators are PDEs on dynamical systems, where, given an initial condition function, the operator represents the map to the solution.

The main distinction lies in the final output: in standard neural networks, the output is given in a discretised fixed Euclidean space. In neural operators, the output is not fixed to a discretisation and therefore the network behaves as an operator acting on the input function. Typically, the input function can't be accessed, and therefore we have to discretise the data. Thus the architecture has to bridge the gap between finite-dimensional

data and infinite-dimensional function spaces [Kovachki et al., 2024]. This is done by modifying the non-operator layer architectures to allow for infinite-dimensional outputs.

2.6.3. Neural operators: Fourier Neural Operators (FNOs)

Fourier Neural Operators (FNOs) (introduced in [Li et al., 2021]) transform the convolutional layer to the operator network space by transforming it to a spectral convolution. Rather than acting on a discretised grid space, the spectral convolution acts on the Fourier space, thus making the operation resolution invariant. The motivation for the spectral convolution is taken from the spectral convolution theorem (the following description is taken from [Duruiseaux et al., 2026]), where given a convolution kernel $\mathcal{K}$ applied on a function parametrised by a k_θ :

$$(\mathcal{K}v)(x) = \int_D k_\theta(x, y)v(y)dy = \int_D k_\theta(x - y)v(y)dy = k_\theta * v, \quad (24)$$

where in the second equality the fact that $\mathcal{K}$ is a convolution was used. This operation, by the spectral convolution theorem, can be written as

$$(\mathcal{K}v)(x) = \mathcal{F}^{-1}(\mathcal{F}(k_\theta) \cdot \mathcal{F}(v))(x) = \mathcal{F}^{-1}(R_\theta \cdot \mathcal{F}(v))(x), \quad (25)$$

where $\mathcal{F}$ is the Fourier transform and $\mathcal{F}(k_\theta)$ was substituted by R_θ , a learnable complex tensor, which, instead of learning the kernel, learns the Fourier coefficients of the kernel. R_θ acts as a linear map between channels for each Fourier mode k , given $v : D \subset \mathbb{R}^{d_{in}} \rightarrow \mathbb{R}^{d_{out}}$, then $R_\theta(k) \in \mathbb{C}^{d_{out} \times d_{in}}$, thus acting as a mixing operator on a given scale size. The collection of matrices $R_\theta(k)$ is truncated to an upper limit on the wave number $|k| < k_{max}$, this is motivated by the higher wave numbers typically being dominated by noise or small fluctuations that shouldn't mix across the whole physical domain. The remaining frequencies are reconstructed through skip, nonlinear, or other mixing layers. In practice, it is discretised, and therefore the Fourier transform can be computed efficiently via the Fast Fourier Transform.

The full architecture is then comprised of, together with the spectral convolutions, point-

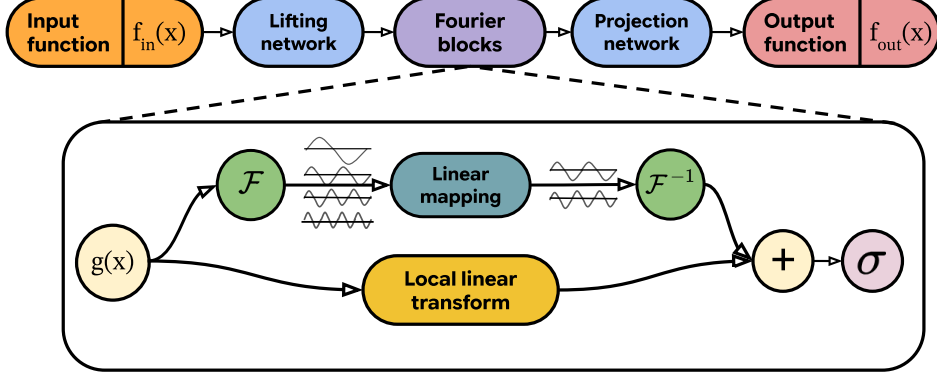


Figure 2.5: FNO full architecture. The discretised input is lifted by the **projection network**. The **spectral blocks** are applied, where a convolution in Fourier space is performed together with a linear transformation. Following the spectral blocks, the state is brought back to its original dimensionality through the **projecting network**.

wise lifting and projecting layers, which raise the low dimensional input into a higher dimensional space and, at the end, lower it back to the original space, respectively, these are typically implemented as MLPs. After the P projection, the FNO blocks are applied, which consist of a spectral convolution together with a pointwise linear transform, usually modelled as convolution with a 1×1 pixel kernel. The output of these layers is then added together and passed through an activation function σ . A scheme of the full architecture can be seen in Figure 2.5. The original FNO architecture has been expanded, for instance as a diffusion model [Liu and Tang, 2025], to general geometries [Li et al., 2024], factorised R_θ matrices [Tran et al., 2023] or with extra operations in the Fourier block such as the application of an integral or local kernels [Liu-Schiaffini et al., 2024].